

Leader election

Contents

1. Problem definition
2. Why leader election is important?
3. Leader election algorithms
 - 3.1. In general networks
 - FloodMax, OptFloodMax
 - 3.2. In synchronous / asynchronous ring
 - LeLann, Chang-Roberts, Hirschberg-Sinclair, Franklin (leader - max identity)
 - Peterson (leader – arbitrary process)
 - 3.3. In anonymous ring
 - Itai-Rodeh

Problem definition

- Elect a unique leader in a network
- All processors have the same local algorithms
- Each computation terminates with one process elected as a leader

Why leader election is important?

- When a leader is selected many other properties of network can be computed:
 - count the number of vertices
 - assign unique identifiers to vertices
 - build a tree
- Possible to execute centralized protocols in decentralized environments
- Recover from token-loss for token-based protocols

Model of distributed system

- Synchronous distributed system:
 - lower and upper bounds on the execution time of processes, messages are received within a known bounded time, drift rates between local clocks have a known bound, global physical time (with a certain precision)
 - predictable behavior
- Asynchronous distributed system:
 - no lower and upper bounds on the execution time of processes, messages are not received within a known bounded time, drift rates between local clocks do not have a known bound no global physical time (logical time is needed)
 - unpredictable behavior

Leader election algorithms

- **communication mechanism**
 - asynchronous
 - **synchronous**
- **process name**
 - **unique identifier**
 - anonymous
- **network topology**
 - ring
 - tree
 - **graph**
- **number of processors**
 - known by the algorithm
 - **not known**

FloodMax
OptFloodMax



strongly connected
graph
processes know
diameter =
the shortest distance
between the two most
distant nodes in
the network

FloodMax algorithm

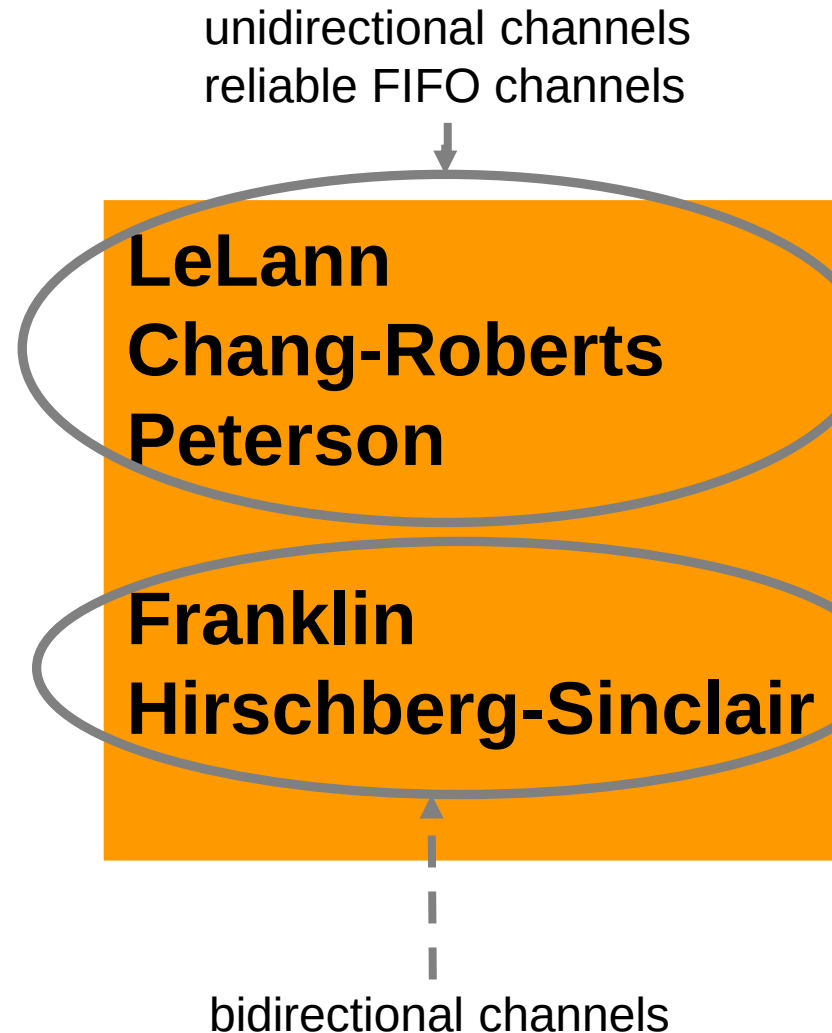
- process maintains a record of the maximum identifier seen so far
- at each round, each process propagates this maximum to all of its outgoing edges
- after *diam* rounds, if the maximum value seen is the process's own identifier, the process elects itself leader, otherwise it is a non-leader
- time complexity: *diam*
- communication complexity : $diam * E$ (E is the number of directed edges in the graph)

OptFloodMax algorithm

- reduced communication complexity
- processes can send their max-uid only when they first learn about them, not at every round
- to reduce the number of messages even more, if a process i receives a new message from a process j that is both an incoming and outgoing neighbor then i need not send a message in the direction of j at the next round

Leader election algorithms

- **communication mechanism**
 - asynchronous
 - synchronous
- **process name**
 - unique identifier
 - anonymous
- **network topology**
 - ring
 - tree
 - graph
- **number of processors**
 - known by the algorithm
 - not known



LeLann algorithm

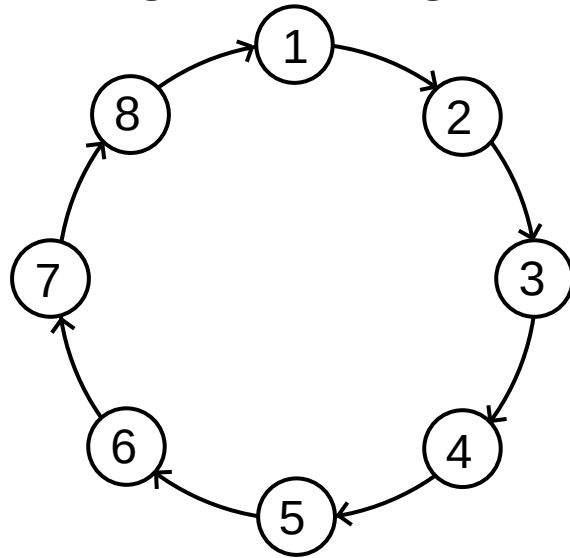
- initially every process is in *unknown* state
- every process sends a message with its own id to its neighbor
- when a process P_i gets a message:
 - $u_i < u_j$: P_i gets the state *non-leader* and pass on u_j
 - $u_i > u_j$: P_i pass on u_j
 - $u_i = u_j$: if the state of P_i is *unknown* then P_i is leader (if the state of P_i is *non-leader* then nothing)
- time complexity: n
- communication complexity: always requires n^2 messages

Chang-Roberts algorithm

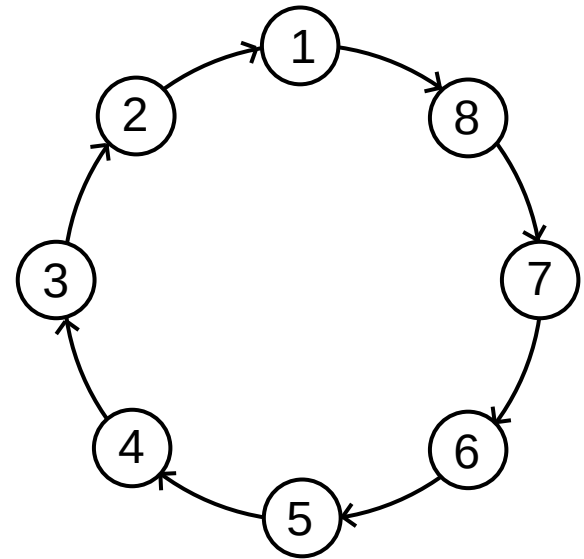
- initially every process is in *unknown* state
- every process sends a message with its own id to its neighbor
- when a process P_i gets a message:
 - $u_i < u_j$: P_i gets the state *non-leader* and pass on u_j
 - $u_i > u_j$: the message with u_j is discarded
 - $u_i = u_j$: P_i is leader

Chang-Roberts algorithm

- time complexity: $2n$
- communication complexity:
 - $2n-1$ messages in the best case
 - $n(n+1)/2$ messages in the worst case
 - $n \log_2(n)$ messages in average case



Best case scenario: $O(n)$



Worst case scenario: $O(n^2)$

Comparison

LeLann

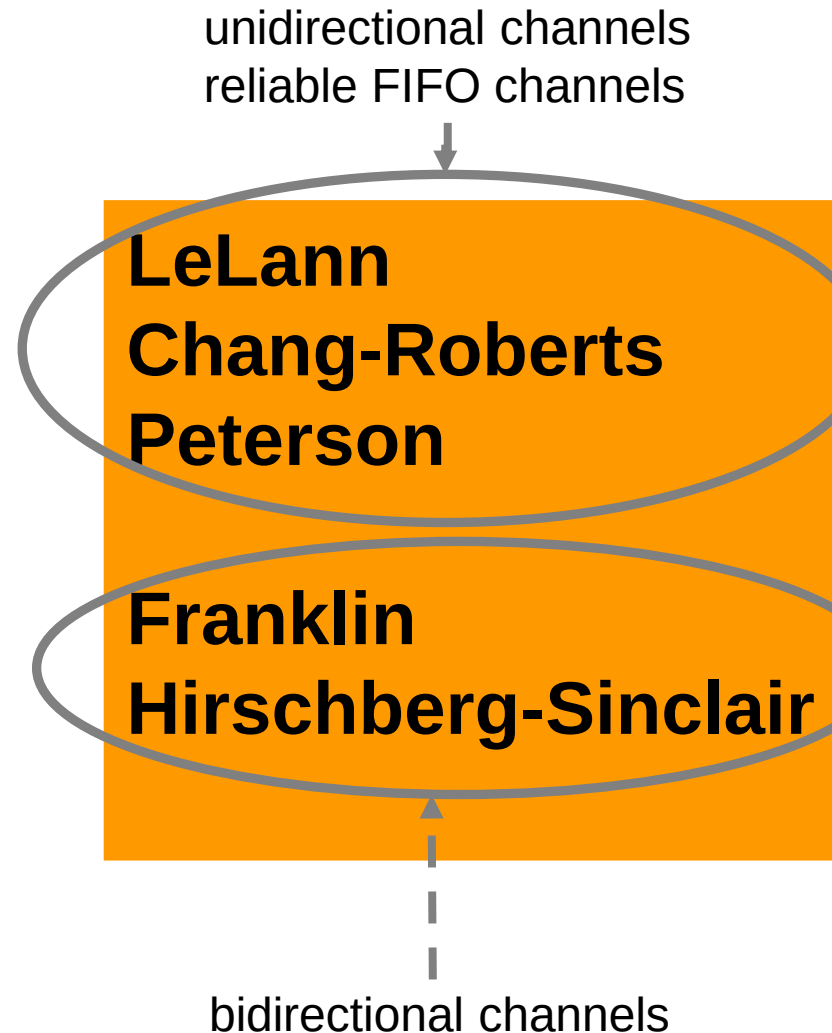
- when a process P_i gets a message:
 - $u_i < u_j$: P_i gets the state *non-leader* and pass on u_j
 - $u_i > u_j$: P_i pass on u_j
 - $u_i = u_j$: if the state of P_i is *unknown* than P_i is leader
- nodes terminate

Chang – Roberts

- when a process P_i gets a message:
 - $u_i < u_j$: P_i gets the state *non-leader* and pass on u_j
 - $u_i > u_j$: the message with u_j is discarded
 - $u_i = u_j$: P_i is leader
- only the leader terminates

Leader election algorithms

- **communication mechanism**
 - asynchronous
 - synchronous
- **process name**
 - unique identifier
 - anonymous
- **network topology**
 - ring
 - tree
 - graph
- **number of processors**
 - known by the algorithm
 - not known

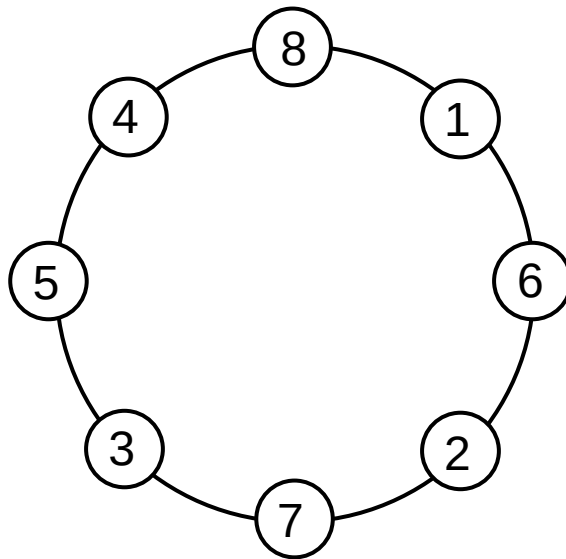


Franklin algorithm

- initially each process is in *active* mode
- every process sends a message with its own id to its neighbors
- when a process P_i gets messages from P_{i-1} and P_{i+1} :
 - if $u_i < u_{i+1}$ or $u_i < u_{i-1}$ P_i becomes passive
 - passive nodes pass on messages
- terminates when a node gets its own identifier
 - only the node with max id get back its own identifier
 - leader announces the other processes
- worst-case message complexity **$O(n \log_2 n)$**

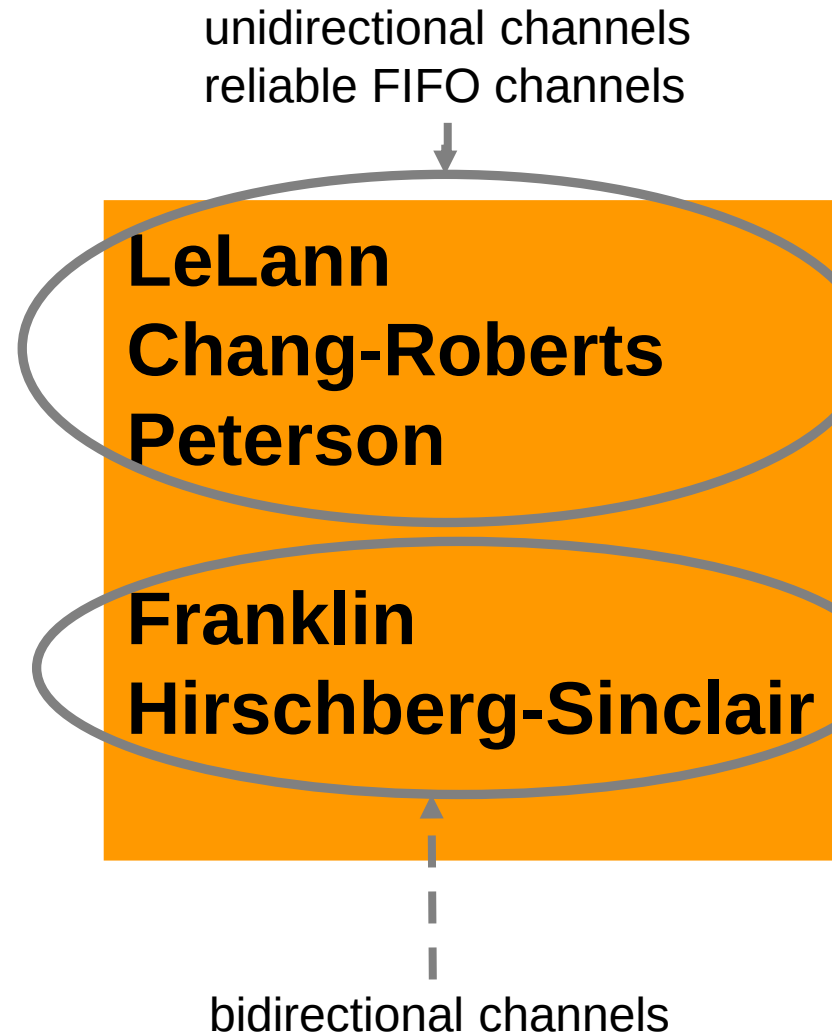
Franklin algorithm

- worst-case message complexity $O(n \log_2 n)$



Leader election algorithms

- **communication mechanism**
 - asynchronous
 - synchronous
- **process name**
 - unique identifier
 - anonymous
- **network topology**
 - ring
 - tree
 - graph
- **number of processors**
 - known by the algorithm
 - not known



Peterson algorithm

- initially each process is in *active* mode
- **phase:** each process P_i sends its identifier to the next and second-next processes in the clockwise direction
 - waits to learn the temporary identifiers from its 2 active predecessors
 - at process P_i if $u_{i-1} > u_{i-2}$ and $u_{i-1} > u_i$, then process P_i remains active, adopting the identifier u_{i-1} of its counterclockwise neighbor as a new “temporary identifier”
 - P_i will be involved in the next phase
 - if the condition doesn't hold, process P_i becomes passive for the remainder of the execution

Peterson algorithm

- algorithm terminates
 - at any phase when process P_i gets an identifier with the same value as its own temporary identifier
 - P_i selected as leader
 - P_i announces the other processes about the leader selection

Peterson algorithm

- algorithm execution is divided into phases
 - in each phase, the number of active processes is reduced by a factor of at least 2, so at most $\log_2 n$ phases exist
- communication complexity
 - during each phase, each process (*active* or *relay*) sends 2 messages
 - at most $\lceil \log n \rceil + 1$ phases exist
 - hence at most $2n(\lceil \log n \rceil + 1)$ messages are sent in every execution of the algorithm → **$O(n \log_2 n)$**

Hirschberg-Sinclair algorithm

- initially each process is in active mode
- every process sends a message with its own id to 2^k distance away $k = 0, 1, \dots$
 - phase k : P_i sends a message containing its identifier u_i in both directions which travels the distance 2^k
 - if both messages return safely, then P_i continues with the next phase
- when P_j receives a message:
 - $u_i < u_j$: P_j discards the message
 - $u_i > u_j$: P_j relays the message
 - $u_i = u_j$: P_j elects itself as a leader
 - algorithm terminates for the selected leader
 - leader announces the other processes
- worst-case message complexity to **$O(n \log_2 n)$**

Leader election algorithms

- **communication mechanism**
 - asynchronous
 - synchronous
- **process name**
 - unique identifier
 - anonymous
- **network topology**
 - ring
 - tree
 - graph
- **number of processors**
 - known by the algorithm
 - not known

process with the largest identity
becomes the leader

LeLann
Chang-Roberts
Hirschberg-Sinclair
Franklin

Peterson

an arbitrary process is elected as leader
not necessarily the process with the
maximum or minimum identifier

Leader election algorithms

Ring Types	Authors	Algorithm Complexity		
		Best Case	Worst Case	Average Case
Unidirectional Ring	LeLann	$O(n^2)$	$O(n^2)$	$O(n^2)$
	Chang-Roberts	$O(n)$	$O(n^2)$	$O(n \log_2 n)$
	Peterson		$O(n \log_2 n)$	
Bidirectional Ring	Franklin		$O(n \log_2 n)$	
	Hirschberg-Sinclair		$O(n \log_2 n)$	

Leader election in anonymous networks

- anonymous network: nodes do not have unique identifiers → nodes are indistinguishable
- deterministic algorithms – impossible for leader election
- probabilistic algorithms for leader election
 - a node picks random identity
 - deterministic algorithm is used to work with pseudo identities
 - deterministic algorithm has to be able to work with duplicate identities

Leader election algorithms

- **communication mechanism**
 - asynchronous
 - synchronous
- **process name**
 - unique identifier
 - **anonymous**
- **network topology**
 - **ring**
 - tree
 - graph
- **number of processors**
 - **known by the algorithm**
 - not known

Itai-Rodeh

Itai-Rodeh algorithm

- probabilistic algorithm
- each process maintains a data structure containing:
 - 1) its identity
 - 2) $\text{state} \in \{\text{active}, \text{passive}, \text{leader}\}$
 - 3) round number
- at the beginning of an election round the active processes send messages containing
 - 1) id and round from the process
 - 2) hop
 - 3) bit

Itai-Rodeh algorithm

- initially: each p_i randomly selects its identity id_i and sends the message $(id_i, 1, 1, \text{true})$
- a passive process p_i receives a message $(id, \text{round}, \text{hop}, \text{bit})$
 - $\text{hop}++$
 - pass on the message
- an active process p_i receives a message $(id, \text{round}, \text{hop}, \text{bit})$
 - 5-step algorithm

Itai-Rodeh algorithm

1. if $\text{hop} = n$ and $\text{bit} = \text{true}$ then the leader is p_i
2. if $\text{hop} = n$ and $\text{bit} = \text{false}$ then p_i takes a new identity and increments round (i.e. moves to the next round) and sends the message $(\text{id}_i^{\text{new}}, \text{round}_i + 1, 1, \text{true})$
3. if $(\text{round}, \text{id}) = (\text{round}_i, \text{id}_i)$ and $\text{hop} < n$ then p_i passes on the message $(\text{id}, \text{round}, \text{hop} + 1, \text{false})$
4. if $(\text{round}, \text{id}) > (\text{round}_i, \text{id}_i)$, the process p_i becomes passive and passes on the message $(\text{id}, \text{round}, \text{hop} + 1, \text{bit})$
5. if $(\text{round}, \text{id}) < (\text{round}_i, \text{id}_i)$ the p_i discards the message

Itai-Rodeh algorithm

- round numbers are essential if the channels are not FIFO
- if the channels are FIFO, round numbers are redundant:
 - if an active process receives a message, then the round number of the message is equal to the round number of the process

Itai-Rodeh algorithm

- Leader election without round numbers
 1. if $\text{hop} = n$ and $\text{bit} = \text{true}$ then the leader is p_i
 2. if $\text{hop} = n$ and $\text{bit} = \text{false}$ then p_i takes a new identity and increments round (i.e. moves to the next round) and sends the message $(\text{id}_i^{\text{new}}, 1, \text{true})$
 3. if $\text{id} = \text{id}_i$ and $\text{hop} < n$ then p_i passes on the message $(\text{id}, \text{hop} + 1, \text{false})$
 4. if $\text{id} > \text{id}_i$, the process p_i becomes passive and passes on the message $(\text{id}, \text{hop} + 1, \text{bit})$
 5. if $\text{id} < \text{id}_i$ the p_i discards the message

Itai-Rodeh algorithm

- Leader election without bits
 - When an active process p_i detects a name clash, meaning that it receives a message with its own identity and hop counter smaller than n , it is not necessary for p_i to wait for its own message to return. Instead p_i can immediately select a new random identity and send a new message. This algorithm is obtained by the previous algorithm. All occurrences of bits are omitted.
1. if $\text{hop} = n$ then the leader is p_i
 2. if $\text{id} = \text{id}_i$ and $\text{hop} < n$ then p_i selects a new random identity and send the message $(\text{id}_i^{\text{new}}, 1)$
 3. if $\text{id} > \text{id}_i$, the process p_i becomes passive and passes on the message $(\text{id}, \text{hop} + 1)$
 4. if $\text{id} < \text{id}_i$ the p_i discards the message

Quiz

- How does Chang-Roberts improve LeLann algorithm?
- What are FIFO channels?